

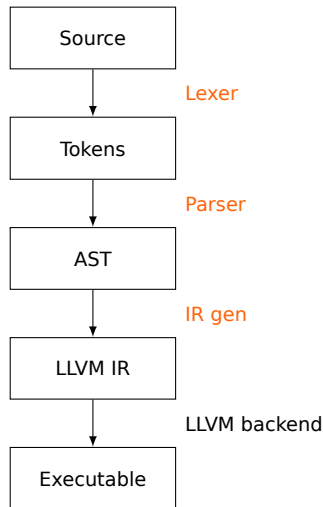
Compiler implementation

Ivan Trepakov

NSU Sys.Pro

What is this course about?

- Building **your own compiler** step by step
 - Use **any language** you like
 - Implement **Lexer**
 - Implement **Parser**
 - Implement **LLVM IR generation**
 - Use LLVM framework as backend
- Incremental evolution of source language
 1. Basic untyped expression language
 2. Control flow constructs
 3. Functions
 4. Primitive type system
 5. Structs and arrays
- Theory and practice together
More compiler theory on elective courses!
- Next semester will re-use developed compiler



Structure

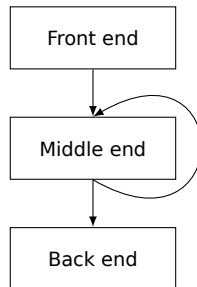
- ~14 weeks, 1.5 classes per week
- Mixed lecture-seminar format
- 5 language extensions to implement
 1. Basic untyped expression language (6 weeks)
 2. Control flow constructs (2 weeks)
 3. Functions (2 weeks)
 4. Primitive type system (2 weeks)
 5. Structs and arrays (2 weeks)
- Final grade is based on implemented extensions
 - **5** — all 5 extensions
 - **4** — 4 out of 5 extensions
 - **3** — 3 out of 5 extensions



<https://nsu-syspro.github.io/courses/translators/>

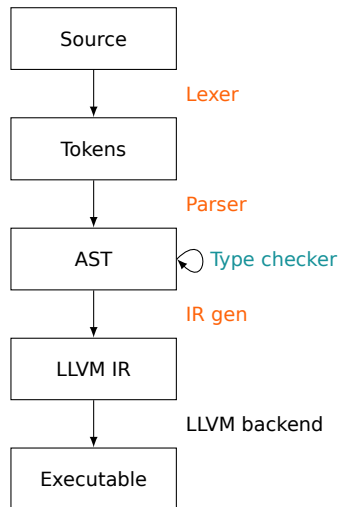
Traditional compilation pipeline

- Front end
 - Lexing (Token stream)
 - Parsing (Abstract Syntax Tree, AST)
 - Type checking (Typed AST)
 - Semantic checking
 - Desugaring
 - Translation to intermediate representation (IR)
- Middle end
 - Machine-independent optimizations and analyses
- Back end
 - Lowering to target arch IR
 - Instruction selection
 - Instruction scheduling
 - Register allocation
 - Machine code generation



Implementation

- We will develop front end for LLVM
 - **Lexer**
 - **Parser**
 - **Type checker** (later)
 - **LLVM IR generation**
- LLVM framework handles middle and back end
 - Optimization
 - Code generation
- All front end stages can be
 - Part of single front end (recommended)
 - Separate programs chained together
- All intermediate formats can be dumped and checked
 - Tokens (JSON)
 - AST (JSON)
 - LLVM IR (.ll or .bc)



Q&A